

Enterprise AI Authentication Research • Part 3 of 7

Part 3

Identity Propagation, OBO & Session Management

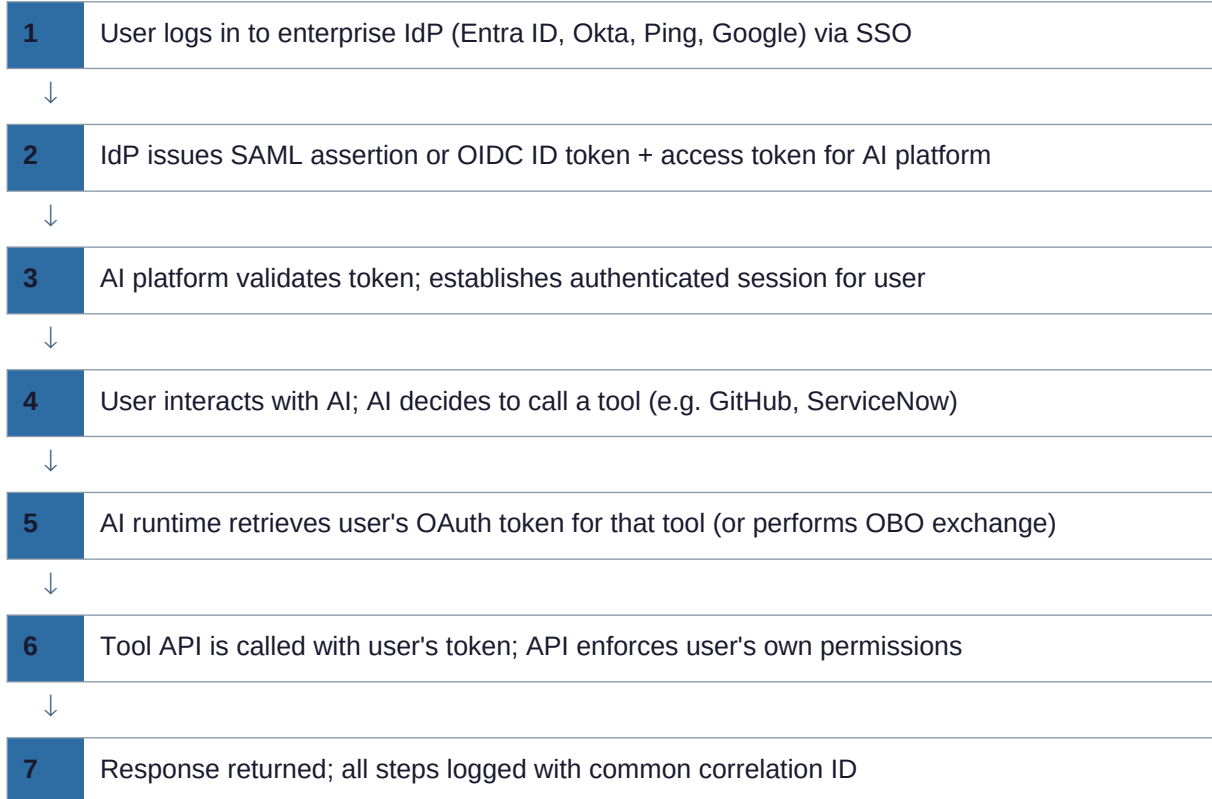
How user identity flows through AI platforms, On-Behalf-Of patterns, long-running sessions, and multi-tool agent orchestration

Enterprise Authentication & Identity Propagation for AI Agents

Part 3A — User Authentication

The foundational question in enterprise AI security is: when an AI agent takes an action, whose identity is presented to the downstream system? This section traces the authentication journey from user login through to the final API call, answering key questions about SSO reuse, session storage, token management, and credential security.

End-to-End Identity Flow



Key Authentication Questions Answered

Authentication Questions — Detailed Answers

Question	Standard Answer	Notes
Is the user authenticated only once?	Yes (SSO + token caching)	User logs in once to IdP; AI platform and tools use issued tokens
Is SSO reused?	Yes	SAML/OIDC session at IdP enables silent re-authentication without password
Is OAuth performed every session?	No (cached tokens + refresh)	First-time consent grants refresh token; subsequent sessions use silent refresh

Question	Standard Answer	Notes
Are refresh tokens stored?	Yes (server-side, encrypted)	Stored in AI platform backend; never sent to browser in production systems
Is browser session reused?	Yes (HttpOnly cookies)	Browser holds session cookie; backend validates and associates with user context
Is PKCE used?	Yes (best practice)	Prevents authorization code interception; required for public clients
Is OAuth Device Flow used?	For CLI/headless agents	GitHub Copilot CLI, AWS CLI use Device Flow; not for web-based AI
Does the connector maintain cookies?	No (token-based)	Modern connectors use bearer tokens; cookies only for web session layer
Is session stored server-side?	Yes (primary storage)	Server-side session for security; browser holds only opaque session ID
Is session stored client-side?	Partially (MSAL cache)	MSAL.js caches tokens in sessionStorage/localStorage; encrypted in production
Are access tokens encrypted?	Yes (in transit and at rest)	TLS in transit; AES-256 or KMS at rest; HSM for high-security deployments
Where are refresh tokens stored?	Server-side encrypted store	Database with encryption; separate from application secrets; rotated regularly
How are secrets rotated?	Automated (OAuth rotation)	Refresh token rotation on each use; access tokens expire and are reissued

Part 3B — Identity Propagation

Identity propagation is the process of carrying the user's verified identity from the AI platform through to every downstream API call. The goal is that each system the AI interacts with sees the user's identity — not the AI platform's service account — so that permissions are correctly enforced and audit logs reflect the actual actor.

What GitHub Sees When an AI Calls Its API

GitHub — What Identity Does the API See?

Scenario	What GitHub API Sees	Identity in Audit Log	Recommended?
AI uses user's OAuth token (delegated)	User's GitHub identity (login, ID)	User login (e.g. john.doe)	YES — full user identity propagation
AI uses GitHub App installation token	App identity + installation context	App name (e.g. my-ai-agent[bot])	Acceptable for org-level tasks; no individual user context
AI uses service account PAT	Service account identity	Service account login (e.g. ai-bot)	AVOID for user-driven actions — masks real actor
AI uses OBO-exchanged token	User's GitHub identity	User login (e.g. john.doe)	YES if GitHub is OIDC-connected to enterprise IdP

Best Practice: For any AI action that creates, modifies, or deletes content in GitHub (commits, PRs, issues, comments), the API call should use the user's own GitHub OAuth token or a user-to-server GitHub App token. Using a shared service account or bot token breaks the audit chain and makes it impossible to attribute changes to individual employees.

Token Exchange Patterns for Identity Propagation

Identity Propagation Patterns

Pattern	Standard	When to Use	Resulting Identity
Direct User Token Pass-Through	OAuth 2.0	When AI platform holds user's token for the target service	Original user identity (sub)
On-Behalf-Of (OBO)	RFC 8693 / MS OBO	Multi-hop: AI platform A → Service B → API C	User identity propagated through the chain
Token Exchange	RFC 8693	Cross-domain token translation (e.g. Entra → GitHub)	User identity in new domain's token format

Pattern	Standard	When to Use	Resulting Identity
JWT Bearer Assertion	RFC 7523	Service asserting user identity without direct user token	Asserted user identity (admin must pre-approve)
SAML Bearer Assertion	OAuth 2.0 SAML 2.0 Profile	Legacy SAML IdP → OAuth 2.0 API	SAML NameID mapped to API user

Part 3C — On-Behalf-Of (OBO) Deep Dive

The OAuth 2.0 On-Behalf-Of (OBO) flow (RFC 8693) enables a middle-tier service (like an AI platform) to exchange an incoming user token for a new token scoped to a downstream service — while preserving the user's identity throughout. This is the gold standard for enterprise AI multi-service identity propagation.

Microsoft Graph OBO Implementation

Microsoft Entra OBO Flow (RFC 8693)



When OBO is Applicable vs. Impossible

OBO Applicability Matrix

Scenario	OBO Applicable?	Alternative	Risk if Skipped
AI calls Microsoft Graph on user's behalf	YES (native Entra OBO)	N/A — use OBO	Risk of elevation with application permissions
AI calls GitHub API using user's GitHub identity	PARTIAL (if OIDC trust configured)	GitHub App user-to-server token	May need separate GitHub OAuth consent
AI calls Salesforce API as user	YES (JWT Bearer with pre-auth)	Web Server OAuth flow per user	Service account access if skipped — audit gap
AI calls on-prem LDAP-authenticated API	NO (LDAP not OAuth-compatible)	Kerberos constrained delegation or SAML	Requires alternative delegation mechanism
AI calls legacy API with Basic Auth only	NO (no token exchange possible)	Privileged service account + extra audit	Identity propagation lost; compliance risk
AI A calls AI B (agent-to-agent)	YES (RFC 8693 token exchange)	Shared context header + JWT	Agent impersonation risk if not verified

RFC 8693 Token Exchange — Key Request Parameters

The following parameters are sent to the IdP token endpoint:

```
POST /token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=urn:ietf:params:oauth:grant-type:token-exchange
&subject_token=
&subject_token_type=urn:ietf:params:oauth:token-type:access_token
&requested_token_type=urn:ietf:params:oauth:token-type:access_token
&audience=
&scope=
&actor_token=
&actor_token_type=urn:ietf:params:oauth:token-type:access_token
```

ServiceNow Delegated Execution

ServiceNow supports two primary models for AI agent delegated execution:

- **OAuth Impersonation:** An OAuth token is issued with the user's identity via a JWT Bearer assertion. The ServiceNow platform sees API calls as coming from the named user.
- **Scripted Delegation:** A Flow Designer flow runs under a service account but logs the original requester's identity using `gs.getUser()` and stored in the `work_notes` or `audit` fields.

ServiceNow OBO Pattern Recommendation

- Use OAuth 2.0 Authorization Code flow to obtain user tokens for interactive AI actions
- Pass the user's sys_user ID in all Flow context variables for audit purposes
- Implement approval workflows (Human-in-the-Loop) for privileged ServiceNow actions
- Log both the acting OAuth user and the originating AI conversation ID in sys_audit

Part 3D — Long-Running Sessions

Enterprise AI conversations may span hours or days — well beyond the lifetime of a typical OAuth access token (60-120 minutes). Production systems must handle token expiry transparently while maintaining security boundaries and respecting session revocation policies.

Token Lifecycle in a Long-Running Agent

Token & Session Lifecycle Components

Mechanism	Purpose	Typical Lifetime	Implementation
Access Token	API authentication	60-120 minutes	Bearer token in Authorization header
Refresh Token	Obtain new access tokens	Hours to 90 days	Stored server-side; used silently
Silent Refresh	Renew access token before expiry	Triggered at 75% lifetime	MSAL/OAuth client library
Continuous Access Evaluation	Revoke token on policy change	Near-real-time (< 30 sec)	CAE-capable resource server
Session Cookie	Browser session continuity	Configurable (30 min – 7 days)	HttpOnly, Secure, SameSite=Strict
Idle Timeout	Terminate inactive sessions	15-30 minutes (enterprise)	Server-side session expiry
Absolute Timeout	Maximum session lifetime	8-12 hours (enterprise)	Forced re-authentication required
Offline Access Scope	Long-lived refresh token permission	90 days (Entra)	Requested at initial consent

How Sessions Survive Common Interruptions

Session Interruption Survival Matrix

Interruption	Survival Mechanism	User Impact	Security Consideration
Expired access token	Silent refresh using refresh token	None (transparent)	Refresh token rotation prevents replay attacks
Browser refresh / page reload	Session cookie + server-side session	None (transparent)	Session ID validated server-side; cookie not readable by JS
Browser close + reopen	Persistent session cookie or re-auth	May require re-auth (config-dependent)	Balance UX vs security based on risk profile

Interruption	Survival Mechanism	User Impact	Security Consideration
Mobile app background / foreground	OS keychain token storage	None (transparent)	Biometric re-auth for high-sensitivity apps
Multiple tabs	Shared session cookie	None	Tab-isolation possible in high-security contexts
Multiple devices	Per-device session (separate token cache)	Independent sessions	CAE ensures policy changes revoke all device sessions
Network disconnect / reconnect	Retry with stored token	Auto-retry on reconnect	Replay prevention via nonce in token

Continuous Access Evaluation (CAE)

CAE is a protocol extension that allows resource servers (e.g. Microsoft Graph, SharePoint) to reject access tokens in near-real-time based on events, even before the token would normally expire. CAE-enabled tokens can have lifetimes of up to 24 hours while remaining revocable within seconds of a policy change.

Events That Trigger CAE Token Revocation

- User password change or reset
- User account disabled or deleted
- Administrator explicitly revokes all refresh tokens (via Entra ID)
- User's location changes and violates Conditional Access policy
- High-risk sign-in detected by Entra ID Identity Protection
- Session revocation via Microsoft Entra admin portal

Part 3E — Multi-Tool Agent Orchestration

A common enterprise AI workflow involves an agent reading from multiple systems in a single conversational turn — for example, reading a GitHub PR, a linked Jira story, a Confluence design page, then updating ServiceNow and posting a Slack summary. This requires five separate OAuth sessions to be managed simultaneously.

Five-Tool Agent Execution Flow



Managing Five OAuth Sessions Simultaneously

Multi-Tool Session Management

Challenge	Solution	Implementation Detail
5 different access tokens	Per-service token store	Encrypted key-value store keyed by (user_id, service_id)
5 different expiry times	Proactive pre-fetching + refresh	Check expiry before each call; refresh if < 5 min remaining
5 different OAuth consent grants	One-time consent per service	User consents once; tokens cached with offline_access scope
Identity propagation per call	User token per service (not shared)	Each tool call uses user's own token for that service
Scope isolation	Minimal scopes per service	GitHub token has only repo:read; SN token only incident:write
Partial failure handling	Graceful degradation	If Confluence fails, continue other tools; report partial results
Token expiry mid-workflow	Just-in-time refresh	Check + refresh before each API call; retry once on 401
Audit correlation	Single conversation_id + span_id per tool	OpenTelemetry trace with parent span = conversation; child spans = tools

Scope Isolation Between Tools

A critical security principle is that OAuth scopes granted for one service must never be passed to another. Each tool call must use the token scoped specifically to that service. The orchestrator maintains a per-service token map and must not allow token reuse across services.

Anti-Patterns to Avoid in Multi-Tool Agents

- NEVER use a single broad-scope service account token for all tool calls
- NEVER share one service's OAuth token with another service
- NEVER cache unencrypted access tokens in conversation context / prompt
- NEVER include access tokens in LLM input or output (prompt injection risk)
- NEVER allow the LLM to directly construct API URLs — use typed tool schemas only
- NEVER skip per-service authorization checks in the name of performance